

服务器前端部署文档

服务器前端部署文档

1. nuxt框架说明

1.1 nuxt项目结构说明

- 1.1.1 pages-路由核心
- 1.1.2 components——组件目录
- 1.1.3 layouts——页面布局
- 1.1.4 middleware——路由拦截
- 1.1.5 composables——逻辑复用
- 1.1.6 api——接口封装层
- 1.1.7 public——静态资源目录
- 1.1.8 server——NuxtServer
 - 1.1.8.1 功能说明
 - 1.1.8.2 在本项目中的作用
 - 1.1.8.3 注意事项
- 1.1.9 utils——工具函数
- 1.1.10 nuxt.config.ts——核心配置文件

1.2 nuxt破解

2. 前端网页说明

- 2.1 页面结构说明（基于pages目录）
- 2.2 页面访问路径说明
- 2.3 页面加载流程（部署视角）
- 2.4 页面与后端交互
- 2.5 前端资源说明
- 2.6 总结（部署重点）

3. 静态打包与部署说明

- 3.1 前端静态打包
- 3.2 打包产物说明
- 3.3 部署方式
- 3.4 Nginx配置（核心）
 - 3.4.1 编辑配置文件：
 - 3.4.2 完整配置
 - 3.4.2.1 Integrated 主前端配置
 - 3.4.2.2 Dataset 前端配置
 - 3.4.2.3 后端接口代理配置

3.5 启动与验证

3.6 常见问题

- 3.6.1 刷新页面后出现 404
- 3.6.2 接口请求失败

1. nuxt框架说明

1.1 nuxt项目结构说明

本项目基于nuxt3构建，其目录结构如下：

```
1  CURTAINWALLWEB-FRONTEND/  
2  |— api/  
3  |— components/  
4  |— composables/  
5  |— layouts/  
6  |— middleware/  
7  |— pages/  
8  |— public/  
9  |— server/  
10 |— utils/  
11 |— app.vue  
12 |— error.vue  
13 |— nuxt.config.ts  
14 |— package.json  
15 |— tsconfig.json  
16 |— tailwind.config.ts
```

Fence 1

1.1.1 pages-路由核心

作用：自动生成前端路由

Nuxt 会根据 pages/ 下的文件结构自动生成 URL 路径

示例：

```
1  pages/  
2  |— index.vue      → /  
3  |— login.vue     → /login  
4  |— user/  
5  |   |— info.vue   → /user/info
```

Fence 2

不需要手写路由配置（无 vue-router 手动配置）

所有页面访问路径由该目录决定

接口联调、Nginx转发路径必须和这里一致

各模块的页面前端请在此处撰写

1.1.2 components——组件目录

存放可复用 UI 组件（如图表、卡片、按钮等）

- 进参与构建，不影响部署结构
- 最终会被打包进.output或dist中

1.1.3 layouts——页面布局

定义页面外壳（如导航栏、侧边栏），主要用于统一页面结构

1.1.4 middleware——路由拦截

- 用于页面访问前的逻辑处理（如登录校验），如JWT校验或者未登录跳转/login
- 若后端有鉴权，需与后端策略保持一致

1.1.5 composables——逻辑复用

- 类似 hooks（Vue3 Composition API）
- 存放请求封装、状态管理等

部署意义：

- 所有 API 请求通常封装在这里
- 接口地址（baseUrl）一般在这里或 config 中定义

1.1.6 api——接口封装层

封装后端所有api请求

如

```
1  api/  
2  |— user.ts  
3  |— stain.ts
```

Fence 3

1.1.7 public——静态资源目录

- 存放不会被打包的静态文件
- 直接映射为网站根路径

会被原样复制到部署目录

常用于：

- 图片
- favicon
- 静态 JSON

1.1.8 server——NuxtServer

`server/` 目录用于编写 Nuxt 内置的服务端接口，其作用类似一个轻量级 Node.js 后端，可直接在前端项目中定义 API。

本项目中的目录结构如下：

```
1  server/
2  |— api/
3  |   |— account/
4  |   |   |— login.post.ts
5  |   |— device/
6  |   |— monitor/
7  |   |— segment/
8  |— middleware/
```

Fence 4

1.1.8.1 功能说明

- `server/api/`：定义接口路由（自动映射为 `/api/*`）
- `*.post.ts`：表示 POST 请求接口
- `middleware/`：服务端中间件（请求拦截、鉴权等）

1.1.8.2 在本项目中的作用

该目录主要用于：

- 本地开发阶段快速提供接口模拟
- 简单业务逻辑处理

但在本项目实际部署架构中：

- 主要后端服务由独立服务提供（Spring Boot / Python 等）
- 仅在开发阶段可以考虑使用

1.1.8.3 注意事项

- 仅在以下方式下生效：

```
1  nuxt build + node .output/server/index.mjs
```

Fence 5

即：需要 Node.js 运行 Nuxt 服务端

- 若采用本项目当前部署方式：

```
1  npm run generate + Nginx 静态部署
```

Fence 6

则：`server/` 目录中的所有接口 **不会生效**

1.1.9 utils——工具函数

存放通用方法（时间处理、格式化等）

1.1.10 nuxt.config.ts——核心配置文件

nuxt.config.ts 是 Nuxt 项目的全局配置文件，用于统一管理前端应用的构建方式、模块依赖以及与后端服务的交互方式。

1. 运行模式

```
1 | ssr: false
```

Fence 7

- 表示当前项目为 SPA（纯前端应用）
- 部署结论：
 - 不需要 Node.js 运行环境
 - 只需构建后将静态文件部署到 Nginx

2. 接口代理（开发 vs 生产）

```
1 |   nitro: {  
2 |     devProxy: {  
3 |       '/api': { target: 'http://8.159.143.133:8000' },  
4 |       '/predict': { target: 'http://47.102.208.89:8007' },  
5 |       '/oss': { target: 'http://8.159.143.133:9000' },  
6 |       '/crackdetection': { target: 'http://110.42.214.164:8001' }  
7 |     }  
8 |   }
```

Fence 8

该配置仅在开发环境生效

用于把前端请求转发到不同后端服务

生产环境必须在 Nginx 中实现同样的转发规则，否则接口无法访问

3. 后端地址配置（环境变量）

```
1 |   runtimeConfig: {  
2 |     public: {  
3 |       apiBase: process.env.NUXT_API_BASE_URL  
4 |     }  
5 |   }
```

Fence 9

定义前端请求的基础 API 地址

部署关键

- 需要在服务器环境中设置：

- `NUXT_API_BASE_URL`

- 或直接通过 Nginx 统一转发 /api

4. 静态资源路径（必须保证）

```
1 app: {  
2   baseUrl: '/',  
3   buildAssetsDir: '/_nuxt/',  
4 }
```

Fence 10

部署关键点：

- Nginx 必须允许访问：

- `/_nuxt/*`

- 否则页面 JS / CSS 无法加载

5. 前端部署人员须知（重点）

- 这是一个 纯前端项目（SPA）
- 构建后只需部署静态文件（无后端依赖）
- 所有 /api、/predict 等请求必须在 Nginx 中做反向代理
- 必须保证 /_nuxt/ 静态资源路径正常访问

1.2 nuxt破解

由于nuxt框架的高级UI组件库需要授权码（付费），所以本小组使用了破解的方法，具体可参考前端仓库中的 `破解nuxt.md` 文件。

2. 前端网页说明

2.1 页面结构说明（基于pages目录）

本目前端页面基于 Nuxt 的 `pages/` 目录自动生成路由，所有网页访问路径均由该目录结构决定。

典型结构如下：

```
1 pages/  
2   └─ index.vue  
3   └─ login.vue  
4   └─ Usermanage.vue  
5   └─ crack/  
6     └─ CrackDetection.vue  
7   └─ vibration/  
8     └─ abnormal.vue
```

Fence 11

2.2 页面访问路径说明

Nuxt 会根据文件路径自动映射为 URL：

文件路径	访问地址	功能说明
pages/index.vue	/	系统首页
pages/login.vue	/login	用户登录页面
pages/user/info.vue	/user/info	用户信息页面
pages/crack/detect.vue	/crack/detect	裂缝检测页面
pages/vibration/monitor.vue	/vibration/monitor	振动监测页面

Table 1

部署重点：

- 所有页面访问必须通过前端入口（Nginx）
- 不允许直接访问 .vue 文件
- 路由由前端控制（SPA）

2.3 页面加载流程（部署视角）

用户访问网页时的流程如下：

浏览器 → Nginx → index.html → JS加载 → 路由匹配 → 页面渲染

具体过程：

1. 用户访问任意路径（如 /login）
2. Nginx 返回统一的 index.html
3. 前端 JS 加载完成
4. Nuxt 根据 URL 匹配 pages/ 中对应页面
5. 渲染对应 Vue 页面

2.4 页面与后端交互

前端页面通过 API 与后端进行数据交互，典型请求路径如下：

1	/login	→ /api/account/login
2	/crack/detect	→ /crackdetection
3	/vibration	→ /predict 或 /history

请求流程：

部署要求:

- 必须在 Nginx 中配置:
 - /api
 - /predict
 - /history
 - /oss
 - /crackdetection

否则页面功能无法正常使用。

2.5 前端资源说明

前端页面依赖以下资源:

- JS 文件 (打包后位于 /_nuxt/)
- CSS 样式文件
- 图片资源 (来自 /public 或 OSS)

2.6 总结 (部署重点)

1. 页面路径由 pages/ 自动生成
2. 所有页面通过 index.html 统一加载 (SPA)
3. 页面功能依赖后端 API, 必须配置反向代理
4. 静态资源路径 /_nuxt/ 必须正常访问

3. 静态打包与部署说明

3.1 前端静态打包

在前端项目根目录执行:

```
1 | npm install
2 | npm run generate
```

执行完成后, Nuxt 会生成静态站点文件, 可直接用于部署。

3.2 打包产物说明

本项目打包后的目录如下：

`/.output/public`

目录结构示例：

```
1 | integrated-frontend/
2 |   └─ index.html
3 |   └─ 200.html
4 |   └─ 404.html
5 |   └─ _nuxt/
6 |   └─ assets/
7 |   └─ login/
8 |   └─ vibration/
9 |   └─ crackdetect/
10 |  └─ resilience/
11 |  └─ userManager/
12 |  └─ ...
```

Fence 15

各文件/目录作用如下：

- index.html：前端应用入口页面
- 200.html：SPA 路由兜底页面（关键）
- 404.html：错误页面
- _nuxt/：打包后的 JS、CSS 等核心资源
- 各业务目录（如 login/、vibration/）：预生成的页面路径

3.3 部署方式

将打包生成的静态文件上传/移动至服务器目录：

`/var/www/integrated-frontend`

该目录用于存放 Integrated 前端主应用的静态资源，例如：

```
1 | index.html
2 | _nuxt/
3 | assets/
4 | login/
5 | vibration/
6 | crackdetect/
7 | resilience/
```

Fence 16

在 Nginx 中，主前端通过以下配置指向该目录：

```

1 location / {
2     root /var/www/integrated-frontend;
3     index index.html;
4     try_files $uri $uri/ /index.html;
5 }

```

Fence 17

其中:

- root /var/www/integrated-frontend; 表示主前端静态文件所在目录
- index index.html; 表示默认入口文件
- try_files *uri*uri/ /index.html; 用于处理前端路由刷新问题

同时, 本项目同时部署了多个前端 (如 dataset 模块), 需分别配置对应路径。

3.4 Nginx配置 (核心)

本项目使用 Nginx 同时完成两类工作:

- 托管前端静态资源
- 将不同路径的接口请求转发到对应后端服务

3.4.1 编辑配置文件:

```

1 vim /etc/nginx/sites-available/proxy

```

Fence 18

3.4.2 完整配置

```

1 server {
2     listen 80;
3     server_name 你的服务器IP;
4
5     root /var/www/curtainwall-frontend;
6     index index.html;
7
8     # 前端路由 (SPA核心)
9     location / {
10         try_files $uri $uri/ /index.html;
11     }
12
13     # =====
14     # 后端接口代理 (必须配置)
15     # =====
16
17     # 用户认证服务
18     location /api/ {
19         proxy_pass http://8.159.143.133:8000/;

```

```

20     proxy_set_header Host $host;
21 }
22
23 # AI推理服务
24 location /predict/ {
25     proxy_pass http://47.102.208.89:8007/;
26     proxy_set_header Host $host;
27 }
28
29 # 历史数据
30 location /history/ {
31     proxy_pass http://47.102.208.89:8007/;
32     proxy_set_header Host $host;
33 }
34
35 # OSS存储
36 location /oss/ {
37     proxy_pass http://8.159.143.133:9000/;
38     proxy_set_header Host $host;
39 }
40
41 # 裂缝检测
42 location /crackdetection/ {
43     proxy_pass http://110.42.214.164:8001/;
44     proxy_set_header Host $host;
45 }
46
47 # 静态资源（Nuxt打包产物）
48 location /_nuxt/ {
49     alias /var/www/curtainwall-frontend/_nuxt/;
50 }
51 }

```

Fence 19

3.4.2.1 Integrated 主前端配置

```

1 location / {
2     root /var/www/integrated-frontend;
3     index index.html;
4     try_files $uri $uri/ /index.html;
5 }

```

Fence 20

该配置用于访问主前端页面。用户访问服务器根路径时，Nginx 会从 /var/www/integrated-frontend 中读取静态文件。

3.4.2.2 Dataset 前端配置

```
1 location /dataset/ {
2     alias /var/www/dataset-frontend/;
3     try_files $uri $uri/ /dataset/index.html;
4 }
```

Fence 21

/dataset/ 是另一个独立前端模块，使用 alias 映射到：

`/var/www/dataset-frontend/`

同时，Dataset 的 CSS、JS、图片和字体资源也分别配置了独立路径：

```
1 location /dataset/css/ {
2     alias /var/www/dataset-frontend/css/;
3 }
4
5 location /dataset/js/ {
6     alias /var/www/dataset-frontend/js/;
7 }
8
9 location /dataset/png/ {
10    alias /var/www/dataset-frontend/png/;
11 }
12
13 location /dataset/fonts/ {
14    alias /var/www/dataset-frontend/fonts/;
15 }
```

Fence 22

3.4.2.3 后端接口代理配置

Nginx 根据不同路径前缀，将请求转发到不同后端服务。

1. 用户认证后端

```
1 location /api/ {
2     rewrite ^/api/(.*)$ /$1 break;
3     proxy_pass http://localhost:8000;
4     include /etc/nginx/proxy_params;
5 }
```

Fence 23

该配置表示访问 /api/xxx 时，会去掉 /api/ 前缀后转发到本机 8000 端口的后端服务。

2. AI 推理服务

```
1 location /predict/ {
2     rewrite ^/predict/(.*)$ /$1 break;
3     proxy_pass http://47.102.208.89:8007;
4     include /etc/nginx/proxy_params;
5 }
```

Fence 24

用于转发 AI 推理相关请求。

3. 历史数据服务

```
1 location /history/ {
2     rewrite ^/history/(.*)$ /$1 break;
3     proxy_pass http://47.102.208.89:8007;
4     include /etc/nginx/proxy_params;
5 }
```

Fence 25

用于转发历史记录相关请求。

4. OSS 服务

```
1 location /oss/ {
2     rewrite ^/oss/(.*)$ /$1 break;
3     proxy_pass http://8.159.143.133:9000;
4     include /etc/nginx/proxy_params;
5 }
```

Fence 26

用于转发文件访问或对象存储相关请求。

5. 裂缝检测服务

```
1 location /crackdetection/ {
2     rewrite ^/crackdetection/(.*)$ /$1 break;
3     proxy_pass http://110.42.214.164:8001;
4     include /etc/nginx/proxy_params;
5 }
```

Fence 27

用于转发裂缝检测相关请求。

3.5 启动与验证

```
1 sudo nginx -t
2 sudo systemctl restart nginx
```

Fence 28

修改 Nginx 配置后，先检查配置是否正确：

```
nginx -t
```

若显示配置正常，则重启 Nginx：

```
若显示配置正常，则重启 Nginx:
```

访问服务器地址：

```
http://8.159.143.133
```

部署人员需要验证以下内容：

1. 主前端页面是否正常加载
2. /dataset/ 页面是否可正常访问
3. 刷新页面后是否仍能正常显示
4. 登录、推理、历史数据、OSS、裂缝检测等接口是否可正常调用

3.6 常见问题

3.6.1 刷新页面后出现 404

原因：前端是 SPA 应用，直接刷新子路由时，Nginx 可能找不到对应物理文件。

解决方式：

```
try_files $uri $uri/ /index.html;
```

3.6.2 接口请求失败

原因可能包括：

- 后端服务未启动
- Nginx 代理路径配置错误
- rewrite 后路径与后端接口不匹配
- proxy_pass 地址或端口错误

解决方式：

- 使用 curl 直接访问后端服务
- 检查 Nginx error log
- 检查对应服务端口是否开放